

# Rule 110: Computing with Simplicity

How an infinite row of binary cells — updated by eight rules — achieves universal computation, and why origami folds follow the same logic.

# What is Rule 110?

## 1 An infinite row of binary cells

Each cell is either **0** (off) or **1** (on) — the simplest possible unit of information.

## 2 Neighborhood-driven updates

At each step, a cell's new state depends on its current state and the states of its two immediate neighbors.

## 3 Eight rules, one number

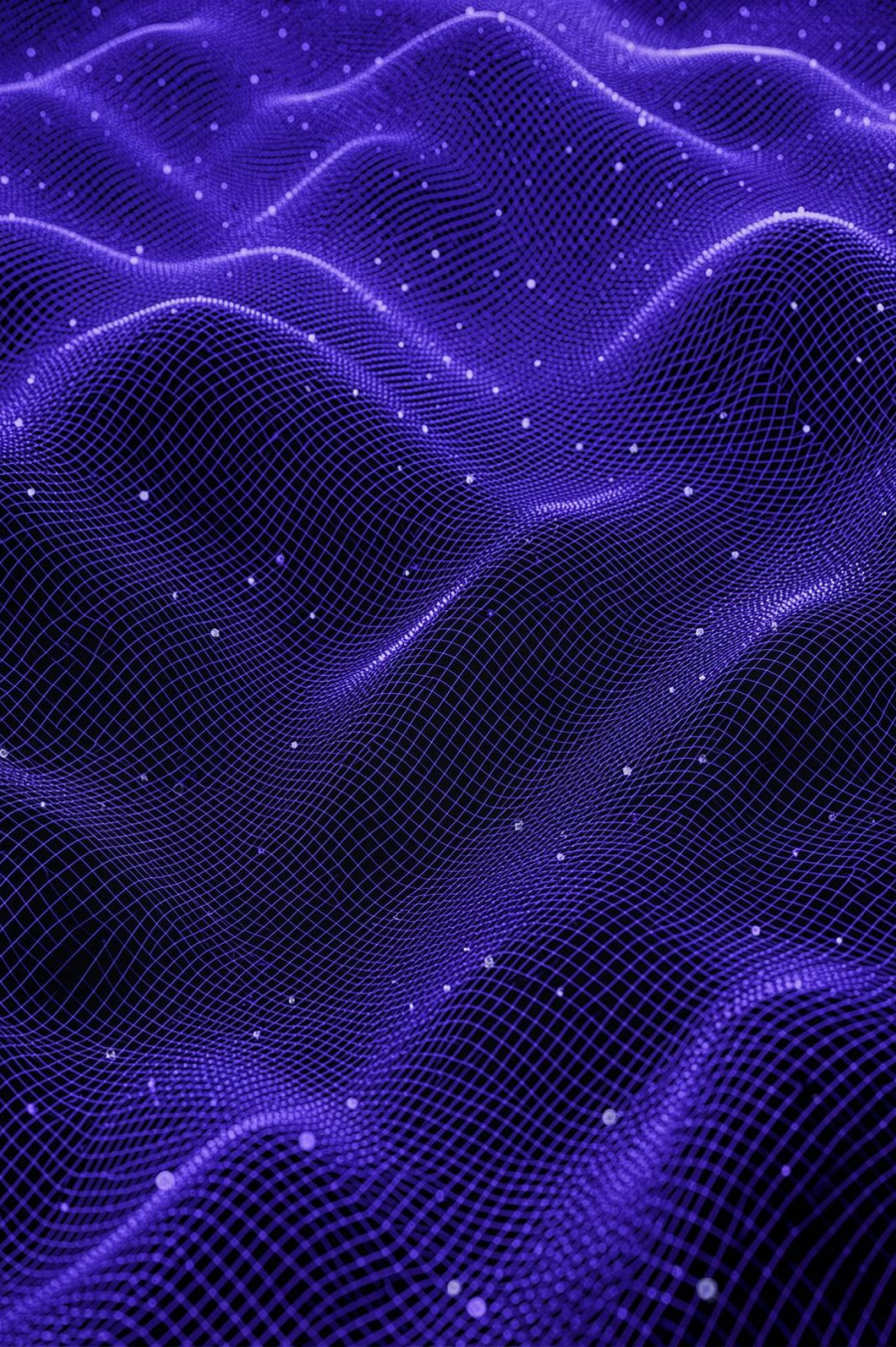
There are 8 possible 3-cell neighborhoods. The binary string **01101110** maps each to its output — decimal 110 gives the rule its name.

# The Rule 110 Update Table

Every possible neighborhood of three cells maps to a single output. Reading the eight outputs right to left gives **01101110** in binary — the number **110** in decimal.

Notice the asymmetry: only **111** and **100** and **000** map to 0. All other patterns sustain or create a live cell — the bias toward 1 is what makes Rule 110 so fertile.

| Neighborhood | Output |
|--------------|--------|
| 111          | 0      |
| 110          | 1      |
| 101          | 1      |
| 100          | 0      |
| 011          | 1      |
| 010          | 1      |
| 001          | 1      |
| 000          | 0      |



# Chaos at the Edge of Order

## Class II – Too Stable

Rules that settle into fixed patterns or simple repetition. Predictable but computationally inert.

## Rule 110 – The Edge

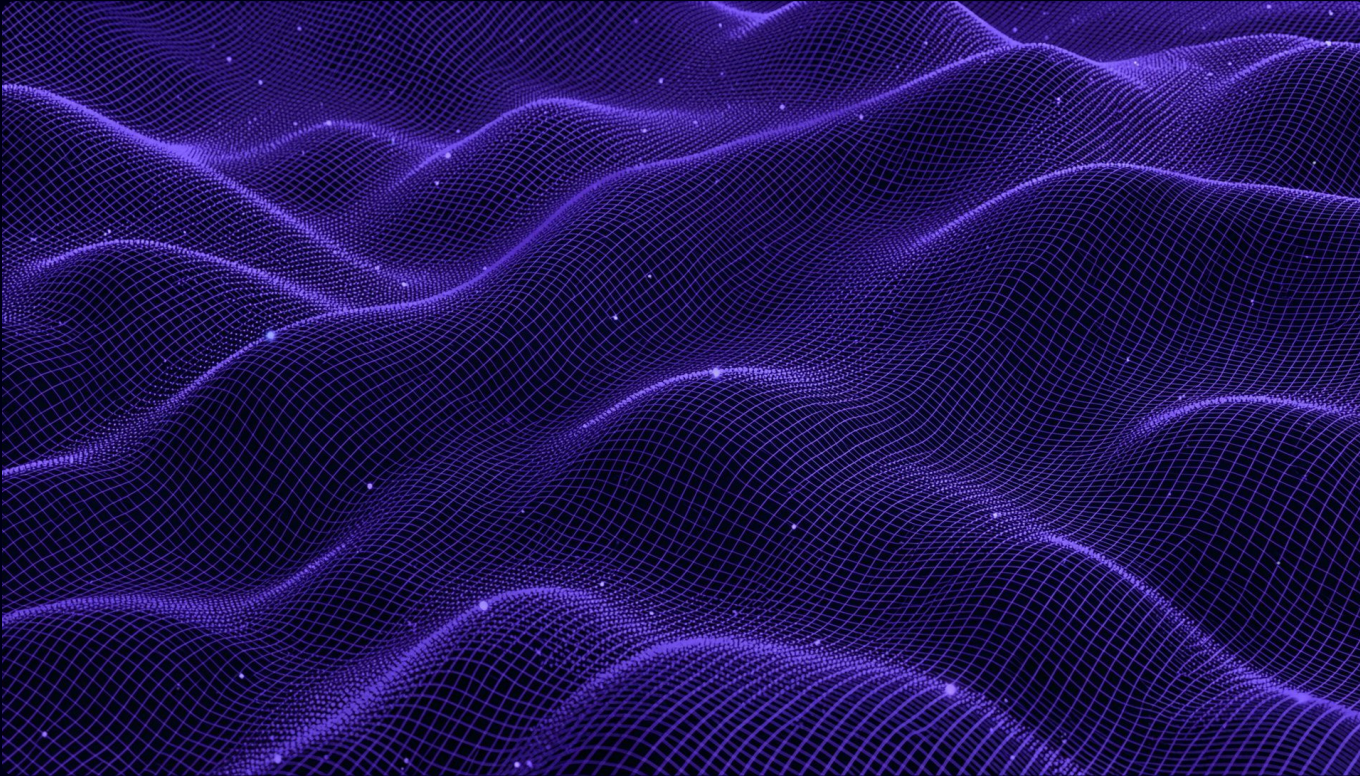
Intricate patterns mixing order and randomness. Complex enough to compute, structured enough to be legible.

## Class III – Too Chaotic

Rules that generate pure noise. Unpredictable and equally inert for practical computation.

Complex computation emerges precisely at this **edge of chaos** — not in stillness, not in noise.

# Gliders: Localized Patterns



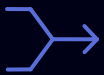
## What is a glider?

A glider is a compact, **self-repeating structure** that travels through the grid at a constant velocity, preserving its shape across generations.

Rule 110 supports multiple glider types — labeled **A through H** — each with its own speed and spatial footprint.

Gliders are the **signals** of Rule 110: localized packets of information moving through an otherwise periodic background.

# Collisions as Logic Gates



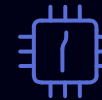
## Gliders as Wires

Moving gliders carry a binary signal — their presence or absence encodes **1** or **0** along a spatial path.



## Collisions as Gates

When two gliders meet, they interact predictably: merging, annihilating, or spawning new gliders — implementing **AND**, **OR**, **NOT**.



## Fredkin & CNOT Gates

Specific collision patterns reproduce reversible **Fredkin gates** and **controlled-NOT gates** — the building blocks of universal logic.

# Building Spatial Patterns

## Eaters

Stable structures that absorb incoming gliders cleanly — acting as signal terminators in a circuit.

## Black Holes

Regions that trap and contain patterns — useful for isolating parts of a computation from interference.

## Flip-Flops

Structures that toggle between two states on each interaction — implementing memory and sequential logic.

Arranged deliberately, these objects form **computational circuits** whose logic is written entirely in space and time.

# Turing Completeness Explained

## What does Turing complete mean?

A system is **Turing complete** if it can simulate any algorithm that any computer can run — given sufficient space and time. It is the theoretical ceiling of computational power.

- ❏ Stephen Wolfram conjectured Rule 110's Turing completeness in 1985. Matthew Cook published the formal proof in **2004**.

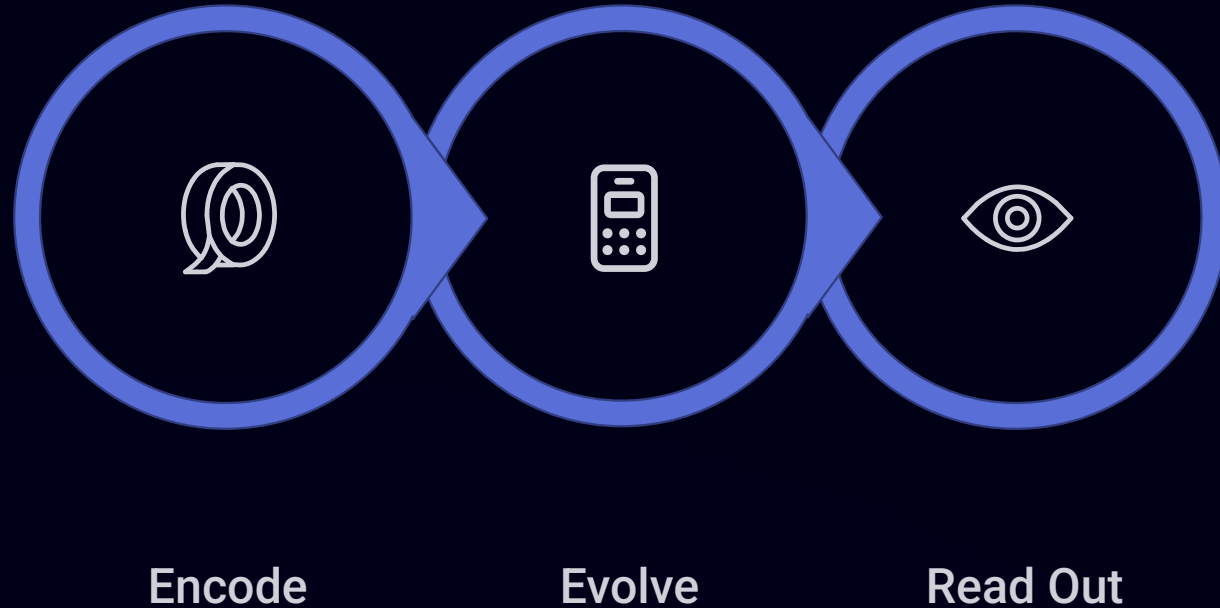
### Any Algorithm

Sorting, searching, arithmetic, simulation — Rule 110 can compute all of it.

### Any Computer

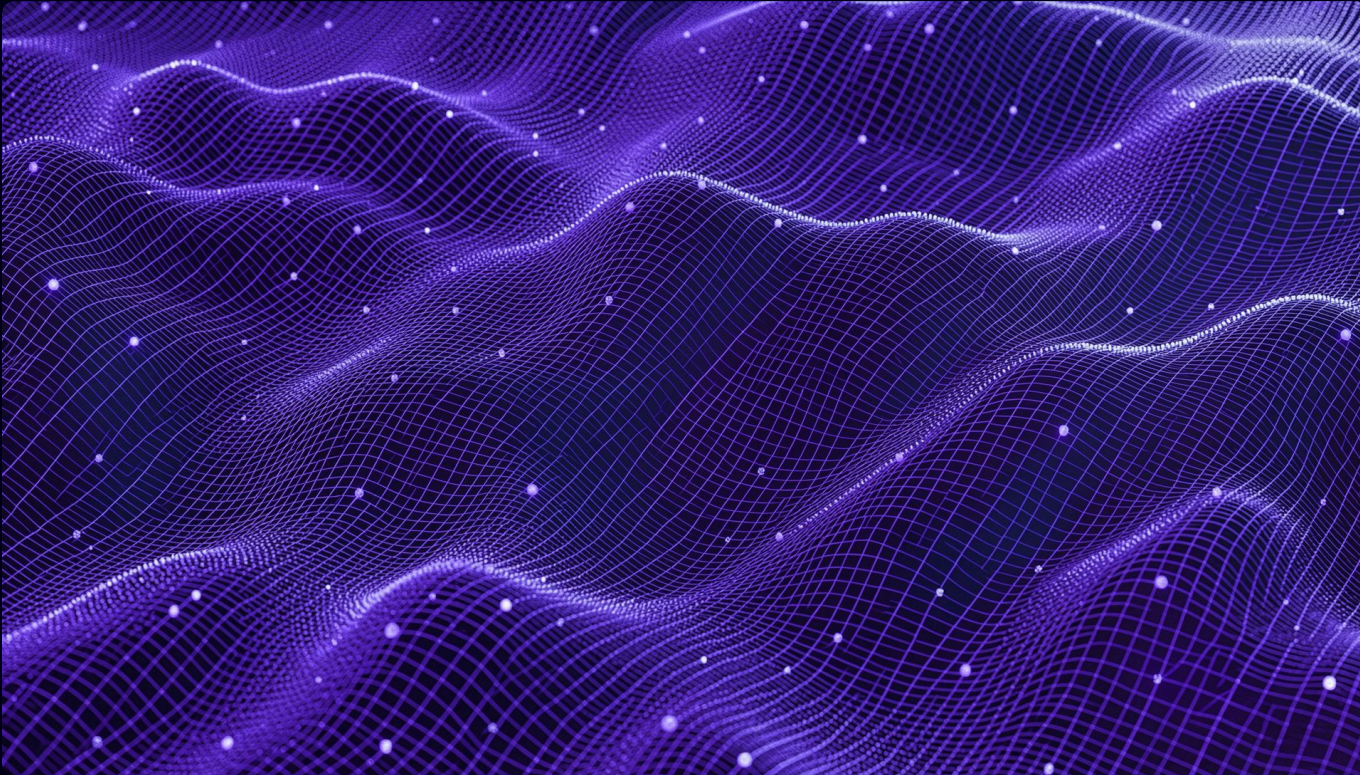
Rule 110 is equivalent in power to a universal Turing machine — the abstract model behind every modern processor.

# How Rule 110 Simulates Computation



Cook's proof is constructive: every component of a Turing machine has a direct glider-based counterpart. The automaton does not approximate computation — it **is** the computation unfolding in space.

# Origami: Folding as Computation



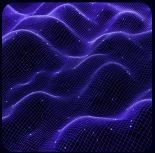
## Flat origami and binary states

In flat origami, paper folds back on itself and stays in a plane. A crease can be folded or left open — a natural **binary choice**.

What matters is the **layer ordering**: which parts of the paper lie above or below others at each intersection. This ordering encodes information.

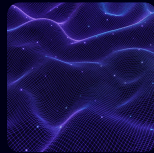
A pleat — two parallel creases — acts as a **bit**: its two possible layer orderings correspond directly to **0** and **1**.

# Origami Logic Gates



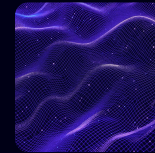
## Gates via Layer Ordering

When two pleats cross, their layer orderings interact. Researchers designed crease patterns where these interactions implement **AND**, **OR**, **NAND**, **NOR**, and **NOT** gates.



## Wires and Propagation

Crease patterns that run in parallel act as **wires**, carrying a layer-ordering signal across the sheet without distortion.



## Mapping to Rule 110

Hull & Zakharevich (2023) showed these origami gates can simulate Rule 110 — establishing that flat origami is itself **Turing complete**.